GRADEOPS

ML Pipeline — Maximum Confidence Grading Architecture

A battle-tested implementation strategy for the Human-in-the-Loop (HITL) AI grading pipeline. Covers OCR strategy, multi-pass grading, confidence scoring, rubric decomposition, plagiarism detection, and LangGraph state design.

1. Core Insight — Skip Separate OCR

The biggest mistake is a 2-step pipeline (OCR text → LLM grades text). Every OCR error compounds into a grading error. Instead, send the raw image directly to a Vision-Language Model that reads handwriting AND grades simultaneously.

Preferred flow:

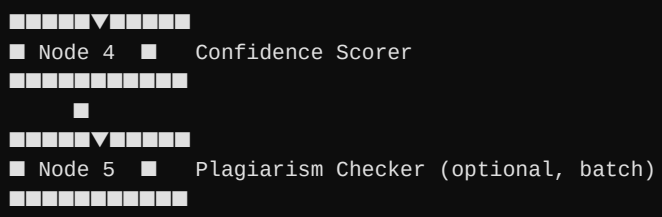
```
[Exam Image]
     ■
     ▼
[VLM sees image + rubric simultaneously]
     ■
     ▼
[Grade  +  Justification  +  Confidence]
```

■ *Qwen2-VL or Claude can read handwriting AND grade in one pass — far fewer error propagation issues than a chained OCR pipeline.*

2. Full Pipeline Architecture (LangGraph)

Each node in the LangGraph graph handles one responsibility. Keeping nodes small makes it easy to swap models or add retries per stage.

```
Image Input
    ■
    ▼
■■■■■■■■■■■■■■■■■■■■
■  Pre-processor  ■   Crop answers, deskew, enhance contrast
■■■■■■■■■■■■■■■■■■■■
         ■
    ■■■■■▼■■■■■
    ■ Node 1  ■   Transcription Agent (Qwen2-VL)
    ■         ■   → Raw handwritten text extraction
    ■         ■   → OCR confidence score per word
    ■■■■■■■■■■■
         ■
    ■■■■■▼■■■■■
    ■ Node 2  ■   Rubric Parser
    ■         ■   → Breaks rubric into atomic sub-criteria
    ■         ■   → e.g. '5 marks = [2 formula, 2 steps, 1 unit]'
    ■■■■■■■■■■■
         ■
    ■■■■■▼■■■■■
    ■ Node 3  ■   Multi-Pass Grader  ← THE KEY NODE
    ■■■■■■■■■■■
         ■
```



3. Multi-Pass Grader — Where Confidence Comes From

Run 3 independent grading passes per answer with different temperatures and/or system personas. The agreement between passes is the primary signal for whether the answer needs human review.

```
# Pseudo-structure
pass_1 = grade(image, rubric, temperature=0.1)   # Strict
pass_2 = grade(image, rubric, temperature=0.3)   # Slightly lenient
pass_3 = grade(image, rubric,
          system="you are a strict examiner...")  # Different persona

confidence = compute_agreement(pass_1, pass_2, pass_3)
```

Confidence Tiers & TA Routing

Agreement Level	Confidence	Score Range	Action
All 3 passes ±5%	HIGH	> 85%	Auto-approve ✓
2 of 3 agree	MEDIUM	50 – 85%	TA reviews
All disagree	LOW	< 50%	TA must grade

■ This is your Human-in-the-Loop trigger — only genuinely ambiguous answers reach the TA dashboard, saving massive review time.

4. Rubric Decomposition Prompt (Critical for Partial Credit)

Don't ask the model 'what's the score out of 10?' — ask it to evaluate each sub-criterion independently. This gives structured partial credit, transparent per-line justification, AND per-criterion confidence.

```
GRADING_PROMPT = """
You are grading a student exam answer.

RUBRIC (decomposed):
{
  "total_marks": 10,
  "criteria": [
    {"id": "C1", "description": "Correct formula stated",      "max_marks": 3},
    {"id": "C2", "description": "Substitution of values",      "max_marks": 3},
    {"id": "C3", "description": "Final answer with correct unit","max_marks": 4}
  ]
}

For EACH criterion, return:
- marks_awarded  (number)
- justification  (1 sentence)
- confidence     (0.0 to 1.0)
```

```
Return ONLY valid JSON.  No explanation outside JSON.
"""
```

5. Model Recommendations

Task	Best Model	Why
Handwriting OCR	Qwen2-VL-7B (local)	Best open-source handwriting, free
Grading logic	Claude Sonnet 4 via API	Best reasoning + structured JSON
Plagiarism / similarity	sentence-transformers MiniLM	Fast cosine similarity, runs locally
Fallback OCR (math)	Nougat	Great for equations & diagrams

■ *For exams with heavy math/diagrams, send the raw image to Claude or Qwen2-VL directly rather than transcribed text — accuracy improves significantly.*

6. Confidence Score Formula

A weighted composite of three signals: inter-pass grade agreement, per-criterion model certainty, and raw OCR legibility.

```
def compute_final_confidence(passes, ocr_conf, rubric_coverage):
    grade_std = std([p.total_score for p in passes])
    avg_criterion_conf = mean([c.confidence for c in passes[0].criteria])

    # Low std = high agreement between passes
    grade_agreement_score = max(0, 1 - (grade_std / max_possible_marks))

    final_confidence = (
        0.50 * grade_agreement_score +  # Do passes agree?
        0.30 * avg_criterion_conf     +  # Is model sure per criterion?
        0.20 * ocr_conf                  # Was handwriting legible?
    )
    return final_confidence
```

7. Plagiarism Detection

Run after all papers for a question are graded. Uses sentence embeddings to detect structurally similar logic, even when surface wording differs.

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer('all-MiniLM-L6-v2')

# Encode all transcribed answers for one question
embeddings = model.encode([paper.transcribed_answer for paper in papers])
similarity_matrix = cosine_similarity(embeddings)

# Flag pairs with similarity > 0.92
suspicious_pairs = [
    (i, j)
    for i in range(len(papers))
    for j in range(i+1, len(papers))
```

```
        if similarity_matrix[i][j] > 0.92
    ]
```

8. LangGraph State Structure

Keep the complete grading context in a single typed dict. This makes it easy to resume, audit, or replay any grading decision.

```
class GradingState(TypedDict):
    image_path:         str
    rubric:             dict
    transcribed_text:   str
    ocr_confidence:     float
    grading_passes:     List[GradingResult]   # 3 independent passes
    final_grade:        float
    final_confidence:   float
    justifications:     List[CriterionResult]
    needs_human_review: bool
    plagiarism_flag:    bool
```

9. Recommended Build Order

Tackle the pipeline in this order to ship a working grader as fast as possible, then layer on confidence and intelligence:

- **Step 1** — Single-pass grader with Claude API + rubric decomposition prompt. Get it working end-to-end first.
- **Step 2** — Add multi-pass grading (3 passes). Confidence scoring kicks in automatically.
- **Step 3** — Integrate Qwen2-VL locally for OCR. Saves API costs at scale while improving handwriting accuracy.
- **Step 4** — Add plagiarism detection last — it operates across papers and needs a batch job, not inline grading.
